Vectors for ML

Vectors are a fundamental concept in machine learning and play an important role in data representation. They are widely used in algorithms for classification, regression, clustering and deep learning. A vector is a mathematical object with magnitude and direction and in machine learning, it represents data in numerical form so that computers can process and analyze it efficiently.

A vector is an ordered list of numbers, called components or elements, where each number represents a feature of a data point. A vector in a two-dimensional plane can be defined as:

$v = (x_1, x_2)$

For example: a vector for someone's height (in cm) and weight (in kg) might be:

v = (170, 65)

In higher dimensions, vectors will have more components, such as:

$v = (x_1, x_2, x_3, ..., x_n)$

where **n** is the number of features of a specific data point.

Code to Create and Print a Vector in NumPy:

```
import numpy as np
vector = np.array([170, 65])
print("Vector:", vector)
```

Output:

```
Vector: [170  65]
```

Scalars, Vectors and Matrices

- **Scalars**: Any single value from our dataset would represent a scalar like integers or floating-point numbers, employed in mathematical computation.
- **Vectors**: Vectors are one-dimensional number arrays that hold several values in a linear sequence.
- **Matrices**: Matrices are two-dimensional arrays of multiple vectors that are placed in rows and columns.

These mathematical structures play an essential role in machine learning models, facilitating effective calculations and data representation.

```
import numpy as np
# Define a 3x3 matrix
mat = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(mat)
```

Output:

```
Matrix:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Vectors in Machine Learning Models

Vectors are used at various points in machine learning models:

1. **Input**: All data such as images, text or sensor readings must be converted into numerical form. These numbers are organized as vectors so models can process them efficiently.
2. **Model**: Machine learning models, especially neural networks, rely on vector and matrix operations. Linear algebra operations like multiplication help update model parameters and improve accuracy.
3. **Outputs**: Model outputs can be numbers, categories or vectors. In tasks like NLP or recommendations, output vectors are used for similarity, clustering or further predictions.

Types of Vectors in Machine Learning

1. Row and Column Vectors

- A **row vector** is a one-dimensional array represented in a row format: (x1, x2, x3)
- A **column vector** is a one-dimensional array represented in a column notation:

[\begin{bmatrix} x_1 \ x_2 \ x_3 \ \vdots \ x_n \end{bmatrix}]

2. Zero Vector

A vector with all elements as zero. Example:

v = (0, 0, 0)

Zero vectors are useful when solving optimization problems and are the origin in vector space.

3. Unit Vector

A vector of magnitude 1. It is frequently used to denote direction:

[\mathbf{u} = \frac{\mathbf{v}}{|\mathbf{v}|}]

where (|\mathbf{v}|) is the magnitude of vector **v**.

4. Sparse and Dense Vectors

- **Sparse Vectors** consist primarily of zeros and are employed in text analysis and recommendation systems.
- **Dense Vectors** consist primarily of non-zero values and are employed in image processing and deep learning.

Importance of Vectors in Machine Learning

1. Feature Representation

Vectors are used to represent data points in numerical form. For example, in natural language processing (NLP), words are translated into word vectors by techniques such as Word2Vec or TF-IDF.

2. Distance and Similarity Measures

Similarity between data points is typically calculated in machine learning by vector distance measures.

3. Transformations and Projections

Vectors enable mathematical transformations such as rotation, scaling and translation. These are employed in methods such as Principal Component Analysis (PCA) to project datasets into lower dimensions.

Vector Operations in Machine Learning

1. Vector Addition and Subtraction

```
import numpy as np
a = np.array([2, 3])
b = np.array([1, 4])
# Perform addition
add = a + b
# Perform subtraction
sub = a - b
print("Addition:", add)
print("Subtraction:", sub)
```

Output:

```
Addition: [3 7]
Subtraction: [ 1 -1]
```

2. Scalar Multiplication

```
import numpy as np
a = np.array([1, 2, 3])
scalar = 3
# Perform scalar multiplication
res = scalar * a
print("Scalar Multiplication:", res)
```

Output:

```
Scalar Multiplication: [3 6 9]
```

3. Dot Product

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
# Compute the dot product
prod = np.dot(a, b)
print("Dot Product:", prod)
```

Output:

```
Dot Product: 32
```

4. Cross Product

```
import numpy as np
c = np.array([1, 2, 3])
d = np.array([4, 5, 6])
# Compute the cross product
prod = np.cross(c, d)
print("Cross Product:", prod)
```

Output:

```
Cross Product: [-3  6 -3]
```

Application of Vectors in Machine Learning

Vectors play an important role in various machine learning algorithms and natural language processing (NLP) techniques.

- **Linear Regression**: Linear regression employs vectors to denote the independent and dependent variable relationship:
  Y = Xw + b where **X** is a feature vector, **w** is a weights vector and **b** is the bias term.
- **Support Vector Machines (SVMs)**: SVMs utilize vector mathematics to identify the best hyperplane separating various classes in classification problems.
- **Neural Networks**: Neural networks store weights, activations and gradients as vectors, making them crucial for deep learning models.
- **Clustering (K-Means Algorithm)**: The K-Means algorithm allocates points to clusters based on vector distances.
- **Word Embeddings in NLP**: Methods such as Word2Vec and GloVe map words to vectors, capturing their semantic meaning.

- **Sentence and Document Embeddings**: Sentences and documents can also be encoded as vectors using techniques like BERT and Doc2Vec.

Linear Combinations

Linear combination involves combining a set of vectors by multiplying each vector by a scalar (a real number) and then adding the results together. For example, if you have vectors $\mathbf{v_1}$ and $\mathbf{v_2}$ and scalars **a** and **b**, the expression $\mathbf{a \times v_1 + b \times v_2}$ is a linear combination of those vectors.

This concept is not limited to just vectors. Linear combinations can also be applied to functions, polynomials and other mathematical entities.

Linear combinations are used in data science and data analysis in the following ways:

Mathematical Definition

Given a set of vectors $\mathbf{v_1, v_2, \ldots, v_n}$ in a vector space, a linear combination of these vectors is an expression of the form:

$$\mathbf{w = c_1v_1 + c_2v_2 + \ldots + c_nv_n}$$

Where $\mathbf{c_1, c_2, \ldots, c_n}$ are scalars (real numbers, complex numbers, etc.).

Example of Linear Combination

Consider two vectors in $\mathbb{R}^2$:

$$\mathbf{v_1 = \begin{pmatrix} 1 \ 2 \end{pmatrix}, v_2 = \begin{pmatrix} 3 \ 4 \end{pmatrix}}$$

A linear combination of $\mathbf{v_1}$ and $\mathbf{v_2}$ would be:

$$\mathbf{w = c_1v_1 + c_2v_2 = c_1 \begin{pmatrix} 1 \ 2 \end{pmatrix} + c_2 \begin{pmatrix} 3 \ 4 \end{pmatrix} = \begin{pmatrix} c_1 + 3c_2 \ 2c_1 + 4c_2 \end{pmatrix}}$$

Properties of Linear Combinations

Some of the common properties of linear combinations are:

- Linearity Property
- Commutative Property
- Associative Property

Linearity Property

Scalar multiplication distributes over addition:

$$\mathbf{a(u + v) = au + av}$$
$$\mathbf{(a + b)u = au + bu}$$

This ensures scaling and addition behave consistently.

Commutative Property

Order of addition does not matter: $\mathbf{v_1 + v_2 = v_2 + v_1}$

In linear combinations: $\mathbf{c_1v_1 + c_2v_2 = c_2v_2 + c_1v_1}$

Associative Property

Grouping does not matter: $(v_1 + v_2) + v_3 = v_1 + (v_2 + v_3)$

In linear combinations: $(c_1v_1 + c_2v_2) + c_3v_3 = c_1v_1 + (c_2v_2 + c_3v_3)$

Real-World Example in Data Science & Data Analysis

Example: House Price Prediction (Linear Regression)

In Linear Regression, the predicted price is a linear combination of features.

Price = w_1(Area) + w_2(Bedrooms) + w_3(Age)

Here:

- Area, Bedrooms, Age → vectors (features)
- w_1, w_2, w_3 → scalars (model weights)

This is exactly a linear combination.

So whenever we build a regression model, we are creating weighted sums of input features.

This is the foundation of:

- Multiple Linear Regression
- Neural Networks (weighted sums)
- Feature engineering

How to Form a Linear Combination

To form a linear combination we can use both vectors and matrices.

Using Vectors

To form a linear combination using vectors, follow these steps:

1. Identify the Vectors: Determine the vectors you want to combine. Let's denote them as $v_1, v_2, \ldots, v_n$.
2. Choose Scalars: Select the scalars (coefficients) that will multiply each vector. Denote these scalars as $c_1, c_2, \ldots, c_n$.
3. Multiply and Add: Multiply each vector by its corresponding scalar and then add the results together.

$w = c_1v_1 + c_2v_2 + \ldots + c_nv_n$

Example:

Given $v_1 = \begin{pmatrix} 1 \ 2 \end{pmatrix}$, $v_2 = \begin{pmatrix} 3 \ 4 \end{pmatrix}$ and scalars $c_1 = 2$ and $c_2 = -1$

The linear combination is:

$w = 2(1, 2) + (-1)(3, 4) = (2, 4) + (-3, -4) = (-1, 0)$

Using Matrices

To form a linear combination using matrices, the process is similar to that of vectors but involves matrix addition and scalar multiplication.

1. Identify the Matrices: Determine the matrices you want to combine. Let's denote them as $A_1, A_2, \ldots, A_n$.
2. Choose Scalars: Select the scalars (coefficients) that will multiply each matrix. Denote these scalars as $c_1, c_2, \ldots, c_n$.
3. Multiply and Add: Multiply each matrix by its corresponding scalar and then add the results together.

$A = c_1A_1 + c_2A_2 + \ldots + c_nA_n$

Example:

Given:
$A_1 = \begin{pmatrix} 1 & 2 \ 3 & 4 \end{pmatrix}$, $A_2 = \begin{pmatrix} 5 & 6 \ 7 & 8 \end{pmatrix}$, scalars $c_1 = 3$ and $c_2 = -2$

Solution:

First, perform the scalar multiplications:
$3 \begin{pmatrix} 1 & 2 \ 3 & 4 \end{pmatrix} = \begin{pmatrix} 3 & 6 \ 9 & 12 \end{pmatrix}$
$-2 \begin{pmatrix} 5 & 6 \ 7 & 8 \end{pmatrix} = \begin{pmatrix} -10 & -12 \ -14 & -16 \end{pmatrix}$

Next, add the resulting matrices:
$A = \begin{pmatrix} 3 & 6 \ 9 & 12 \end{pmatrix} + \begin{pmatrix} -10 & -12 \ -14 & -16 \end{pmatrix} = \begin{pmatrix} -7 & -6 \ -5 & -4 \end{pmatrix}$

Matrices and Matrix Arithmetic for Machine Learning

In machine learning data often comes in multi-dimensional arrays making matrices an ideal way to handle such inputs. Understanding matrix arithmetic is essential because many machine learning algorithms including linear regression, neural networks and dimensionality reduction techniques, rely heavily on matrix operations.

A matrix is a two-dimensional array of numbers arranged in rows and columns. Each element in a matrix is represented as **a[i][j]** where:

- **i**: row number
- **j**: column number

The order of a matrix is expressed as **m x n** where:

- **m**: number of rows
- **n**: number of columns

Defining a Matrix in Python

In Python the NumPy library is commonly used to work with matrices. Matrices can be created using the `numpy.array()` function:

```
import numpy as np
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(matrix)
```

Output:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Here we see a list of lists which is a two-dimensional array having 3 rows and 3 columns. The order of the above matrix is **3 x 3**.

Matrix Arithmetic

Matrix arithmetic involves performing mathematical operations on matrices. These operations are fundamental in machine learning for manipulating and transforming multi-dimensional data efficiently.

1. Matrix Addition

Matrix addition combines two matrices of the same order by adding their corresponding elements. Addition is done using the `+` operator in Python. It is only possible when both matrices have the same number of rows and columns.

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
C = A + B
print(C)
```

Output:

```
[[ 6  8]
 [10 12]]
```

Calculation:

- C[0][0] = 1 + 5 = 6
- C[0][1] = 2 + 6 = 8
- C[1][0] = 3 + 7 = 10
- C[1][1] = 4 + 8 = 12

2. Matrix Subtraction

Matrix subtraction subtracts the corresponding elements of two matrices of the same order. Subtraction is done using the `-` operator in Python. It is only possible when both matrices have the same number of rows and columns.

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[0, 1], [2, 1]])
C = A - B
print(C)
```

Output:

```
[[1 1]
 [1 3]]
```

Calculation:

- C[0][0] = 1 - 0 = 1
- C[0][1] = 2 - 1 = 1
- C[1][0] = 3 - 2 = 1
- C[1][1] = 4 - 1 = 3

3. Matrix Division

Matrix division divides the corresponding elements of two matrices of the same order. Division is done using the `/` operator for float division or `//` operator for integer division in Python.

```
import numpy as np
A = np.array([[4, 2], [6, 8]])
B = np.array([[2, 2], [3, 4]])
C = A // B
print(C)
```

Output:

```
[[2 1]
 [2 2]]
```

Calculation:

- C[0][0] = 4 // 2 = 2
- C[0][1] = 2 // 2 = 1
- C[1][0] = 6 // 3 = 2
- C[1][1] = 8 // 4 = 2

4. Matrix Multiplication (Dot Product)

Matrix multiplication multiplies rows of the first matrix with columns of the second matrix. Multiplication is done using the `.dot()` method or the `@` operator in Python. It requires that the number of columns in the first matrix equals the number of rows in the second matrix.

Note: Matrices do not satisfy the commutative property (AB ≠ BA).

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
C = A.dot(B)
print(C)
```

Output:

```
[[19 22]
 [43 50]]
```

Calculation:

- C[0][0] = 1*5* + *2*7 = 19
- C[0][1] = 1*6* + *2*8 = 22
- C[1][0] = 3*5* + *4*7 = 43
- C[1][1] = 3*6* + *4*8 = 50

5. Vector Multiplication

A vector is a one-dimensional array that can be either a row or a column. To multiply a matrix by a vector, the number of columns in the matrix must equal the number of rows in the vector.

```
import numpy as np
A = np.array([[1, 2], [1, 1]])
V = np.array([[1], [1]])
C = A.dot(V)
print(C)
```

Output:

```
[[3]
 [2]]
```

Calculation:

- C[0][0] = 1*1* + *2*1 = 3
- C[1][0] = 1*1* + *1*1 = 2

6. Scalar Multiplication

When we multiply a matrix with a scalar then it is multiplied with each and every element in the matrix. The order remains same even after multiplying the matrix with scalar.

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
b = 2
print(A * b)
```

Output:

```
[[2 4]
 [6 8]]
```

Calculation:

- C[0][0] = 1 * 2 = 2
- C[0][1] = 2 * 2 = 4
- C[1][0] = 3 * 2 = 6
- C[1][1] = 4 * 2 = 8

Properties

- Matrices must have the same dimensions for addition, subtraction, and division.
- Matrix multiplication requires that the number of columns in the first matrix equals the number of rows in the second matrix.
- Matrix multiplication is **not commutative**: AB ≠ BA.
- Scalar multiplication preserves the matrix order.

Applications in Machine Learning

Matrices are essential in machine learning for handling multi-dimensional data. Key applications include:

- **Linear Regression**: Uses matrix operations for coefficient computation and predictions (e.g., **Y = XW + b**).
- **Neural Networks**: Weight matrices, forward propagation, and backpropagation rely heavily on matrix multiplication.
- **Dimensionality Reduction**: Techniques like PCA use matrix decompositions and eigenvalue calculations.
- **Data Representation**: Images, feature vectors, and batch data are stored and processed as matrices.

Matrix arithmetic enables efficient computation and transformation of large datasets in ML algorithms.

Linear Mapping

Linear mapping is a mathematical operation that transforms a set of input values into a set of output values using a linear function. It is often used as a preprocessing step to transform the input data into a more suitable format for analysis. It can also be used as a model in itself, such as in linear regression or linear classifiers.

The linear mapping function can be represented as follows:

y = Wx + b

where

- **x** is the input vector,

- **W** is the weight matrix,
- **b** is the bias vector,
- **y** is the output vector.

The weight matrix and bias vector are learned during the training process.

Let **V** and **W** be vector spaces over a field **K**. A function **f: V → W** is called a **linear map** if, for any vectors **u, v ∈ V** and a scalar **c ∈ K**, the following conditions hold:

- **Additive property**:
  f(u + v) = f(u) + f(v)

- **Homogeneity property**:
  f(c · u) = c · f(u)

Zero/Identity Transformation

A linear transformation **T: V → V** from a vector space into itself is called a **Linear operator**.

- **Zero-Transformation**: For a transformation **T: V → W** is called zero-transformation if:
  T(v) = 0 ∀ v ∈ V

- **Identity-Transformation**: For a transformation **T: V → V** is called identity-transformation if:
  T(v) = v ∀ v ∈ V

Properties of Linear Transformation

Let **T: V → W** be the linear transformation where **u, v ∈ V**. Then, the following properties are true:

- **T(0) = 0**
- **T(-v) = -T(v)**
- **T(u - v) = T(u) - T(v)**

If $\mathbf{v = c_1v_1 + c_2v_2 + \ldots + c_nv_n}$,
Then $\mathbf{T(v) = c_1T(v_1) + c_2T(v_2) + \ldots + c_nT(v_n)}$

Linear Transformation of Matrix

Let **T** be an **m × n** matrix. The transformation $\mathbf{T: \mathbb{R}^n \to \mathbb{R}^m}$ is a linear transformation if:

T(v) = A v

Zero and Identity Matrix operations:

- An **m × n** matrix is a **zero matrix**, corresponding to zero transformation from $\mathbb{R}^n \to \mathbb{R}^m$.
- An **n × n** matrix is the **Identity matrix** $\mathbb{I}_n$, corresponding to identity transformation from $\mathbb{R}^n \to \mathbb{R}^n$.

Matrix representation:

[\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \ a_{21} & a_{22} & \dots & a_{2n} \ \vdots & \vdots & \ddots & \vdots \ a_{m1} & a_{m2} & \dots & a_{mn}

\end{bmatrix} \begin{bmatrix} v_1 \ v_2 \ \vdots \ v_n \end{bmatrix}

\begin{bmatrix} a_{11}v_1 + a_{12}v_2 + \dots + a_{1n}v_n \ \vdots \ a_{m1}v_1 + a_{m2}v_2 + \dots + a_{mn}v_n \end{bmatrix}]

Example

Let's consider the linear transformation from $\mathbb{R}^2 \to \mathbb{R}^3$ such that:

[L\begin{pmatrix} v_1 \ v_2 \end{pmatrix} = \begin{pmatrix} v_2 \ v_1 - v_2 \ v_1 + v_2 \end{pmatrix}]

Verification that it is a linear transformation:

1. **Homogeneity**:
   L(c v) = c · L(v)
   [L\left(c \begin{bmatrix} v_1 \ v_2 \end{bmatrix}\right) = \begin{bmatrix} c v_2 \ c v_1 - c v_2 \ c v_1 + c v_2 \end{bmatrix} = c \begin{bmatrix} v_2 \ v_1 - v_2 \ v_1 + v_2 \end{bmatrix} = c L(\vec{v})]

2. **Additivity**:
   L(v + w) = L(v) + L(w)
   Let $\mathbf{v = [v_1, v_2]^T}$, $\mathbf{w = [w_1, w_2]^T}$
   $\mathbf{v + w = [v_1 + w_1, v_2 + w_2]^T}$
   [L(\vec{v} + \vec{w}) = \begin{bmatrix} v_2 + w_2 \ (v_1 + w_1) - (v_2 + w_2) \ (v_1 + w_1) + (v_2 + w_2) \end{bmatrix} = L(\vec{v}) + L(\vec{w})]

It proves that the above transformation is a **linear transformation**.

Note: Examples of **non-linear** transformations include trigonometric transformations and polynomial transformations.

Kernel and Range Space

Kernel (Null Space)

Let **T: V → W** be a linear transformation. Then ∀ **v ∈ V** such that **T(v) = 0** is the **kernel space** of **T**. It is also known as the **null space** of **T**.

- The kernel space of zero transformation is **{0}**.
- The kernel space of identity transformation is **{0}**.

The dimension of the kernel space is known as **nullity** or **null(T)**.

Range Space

Let **T: V → W** be a linear transformation. Then the set of all **T(v)** for **v ∈ V** is the **range space** of **T**. The range space is always non-empty because **T(0) = 0**.

The dimension of the range space is known as **rank(T)**.

Rank-Nullity Theorem:

null(T) + rank(T) = dim(V) = n

Linear Transformation as Rotation

Some transformation operators rotate a vector by an angle **θ**.

The linear transformation **T:** $\mathbb{R}^2 \rightarrow \mathbb{R}^2$ given by the matrix:

$$A = \begin{bmatrix} \cos\theta & -\sin\theta \ \sin\theta & \cos\theta \end{bmatrix}$$

rotates every vector anti-clockwise about the origin by angle **θ**.

Let $\mathbf{v = [r \cos \alpha, r \sin \alpha]^T}$

Then

$$T(v) = A \cdot v = \begin{bmatrix} r \cos(\theta + \alpha) \ r \sin(\theta + \alpha) \end{bmatrix}$$

which is the original vector rotated by **θ**.

Linear Transformation as Projection

A linear transformation **T:** $\mathbb{R}^3 \rightarrow \mathbb{R}^3$ is given by:

$$T = \begin{bmatrix} 1 & 0 & 0 \ 0 & 1 & 0 \ 0 & 0 & 0 \end{bmatrix}$$

For a vector $\mathbf{v = (x, y, z)}$,
$\mathbf{T(v) = (x, y, 0)}$

This is the **orthogonal projection** of the original vector onto the xy-plane.

Advantages of Linear Mapping

- **Simplicity**: It is a simple and easy-to-understand mathematical operation, making it an attractive choice for many machine learning applications.
- (The article cuts off here in the available content, but the main sections are fully covered above.)

Solving Linear Equations Data Science

Linear Algebra is important in Data Science as it helps represent and process data efficiently, especially for high-dimensional datasets. It also helps in understanding relationships between variables. This is useful in the following ways:

- Efficient Data Representation: Organizes data in matrix form
- Find Relationships: Identifies important variables and patterns
- Supports ML Algorithms: Forms the basis of many machine learning methods

Detecting Linear Relationships Between Attributes

Linear relationships among attributes are identified using the concepts of null space and nullity. These concepts help determine whether variables are linearly dependent and whether some attributes can be expressed as combinations of others.

A generalized system of linear equations is represented as:

$$\mathbf{A\,x = b}$$

Where:

- **A** is an **m × n** matrix of coefficients

- **x** is an **n × 1** vector of unknown variables
- **b** is an **m × 1** dependent variable vector
- **m** represents the number of equations
- **n** represents the number of variables

Rank Conditions in Linear Systems

In general there are three cases that need to be understood when analyzing linear systems. These cases depend on the rank of the matrix and describe how rows and columns relate to one another. Each case is considered independently.

Case 1: m = n

The solution for this type of linear equation if **A** is a full rank matrix having determinant of **A** is equal to 0 will be:

A x = b
x = A^{-1} b

1. Unique Solution

Consider the given matrix equation

[\begin{bmatrix} 1 & 3 \ 2 & 4 \end{bmatrix} \begin{bmatrix} x_1 \ x_2 \end{bmatrix}

\begin{bmatrix} 7 \ 10 \end{bmatrix}]

- |A| is not equal to zero
- rank(A) = 2: no. of columns this implies that A is full rank

[x = \begin{bmatrix} 1 & 3 \ 2 & 4 \end{bmatrix}^{-1} \begin{bmatrix} 7 \ 10 \end{bmatrix} = \begin{bmatrix} 1 \ 2 \end{bmatrix}]

Therefore, the solution for the given example is **(x_1, x_2) = (1, 2)**

2. Infinite Solutions

Consider the given matrix equation

[\begin{bmatrix} 1 & 2 \ 2 & 4 \end{bmatrix} \begin{bmatrix} x_1 \ x_2 \end{bmatrix}

\begin{bmatrix} 5 \ 10 \end{bmatrix}]

- |A| is not equal to zero
- rank(A) = 1, nullity = 1

Checking consistency: Row 2 is twice Row 1 so the system has only one linearly independent equation. Since there are two variables but only one independent equation, the system is consistent and has infinitely many solutions.

$x_1 + 2x_2 = 5$

We can choose any value for x_2. For each choice of x_2, there is a corresponding x_1. Therefore, there are infinitely many solutions to the system.

3. No Solution

Consider the given matrix equation:

[\begin{bmatrix} 1 & 2 \ 2 & 4 \end{bmatrix} \begin{bmatrix} x_1 \ x_2 \end{bmatrix}

\begin{bmatrix} 5 \ 9 \end{bmatrix}]

- |A| is not equal to zero
- rank(A) = 1
- nullity = 1

Checking consistency:
$2(x_1 + 2x_2) = 2x_1 + 4x_2 = 10 \neq 9$

We cannot find the solution to (x_1, x_2).

Case 2: m > n

- In this case the number of variables or attributes is less than the number of equations.
- Here not all the equations can be satisfied.
- So it is sometimes termed as a case of no solution.
- But we can try to identify an appropriate solution by viewing this case from an optimization perspective.

An optimization perspective

Instead of finding an exact solution to the system **A x = b**, we can find an **x** that minimizes the difference **A x - b**.

Let the error vector be:
$\mathbf{e = A x - b}$

We can minimize all the errors collectively by minimizing $\sum \mathbf{e_i^2}$

The optimization problem becomes:

$$\mathbf{\min[(A x - b)^T (A x - b)]}$$

Differentiating and setting the gradient to zero:

$$\mathbf{2(A^T A) x - 2 A^T b = 0}$$

Assuming that all the columns are linearly independent:

$$\mathbf{x = (A^T A)^{-1} A^T b}$$

Note: While this solution **x** might not satisfy all the equations but it will ensure that the errors in the equations are collectively minimized. This is known as the **Least Squares Solution**.

Example

Consider the given matrix equation:

[\begin{bmatrix} 1 & 0 \ 2 & 0 \ 3 & 1 \end{bmatrix} \begin{bmatrix} x_1 \ x_2 \end{bmatrix}

\begin{bmatrix} 1 \ -0.5 \ 5 \end{bmatrix}]

Here **m=3**, **n=2**

Using the optimization concept:

[\begin{bmatrix} x_1 \ x_2 \end{bmatrix} = \begin{bmatrix} 0.2 & -0.6 \ -0.6 & 2.8 \end{bmatrix} \begin{bmatrix} 1 \ -0.5 \ 5 \end{bmatrix} = \begin{bmatrix} 0 \ 5 \end{bmatrix}]

Therefore, the solution is $(x_1, x_2) = (0, 5)$

Important Point: If we have more equations than variables, we can always use the least squares solution $\mathbf{x} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}$.
$(\mathbf{A}^T \mathbf{A})^{-1}$ exists if the columns of **A** are linearly independent.

Case 3: m < n

- This case deals with more number of attributes or variables than equations.
- Here we can obtain multiple solutions for the attributes.
- This is an infinite solution case.
- We will see how we can choose one solution from the set of infinite possible solutions.

Given the optimization problem:
$\min [\frac{1}{2} \mathbf{x}^T \mathbf{x}]$ such that $\mathbf{A}\,\mathbf{x} = \mathbf{b}$

We can define a Lagrangian function:

$\min [f(\mathbf{x}, \lambda)] = \min [\frac{1}{2} \mathbf{x}^T \mathbf{x} + \lambda^T (\mathbf{A}\,\mathbf{x} - \mathbf{b})]$

Eigenvalues and Eigenvectors

Eigenvalues and eigenvectors are fundamental concepts in linear algebra, used in various applications such as matrix diagonalization, stability analysis, and data analysis (e.g., Principal Component Analysis). They are associated with a square matrix and provide insights into its properties.

Eigenvalues

Eigenvalues are unique scalar values linked to a matrix or linear transformation. They indicate how much an eigenvector gets stretched or compressed during the transformation. The eigenvector's direction remains unchanged unless the eigenvalue is negative, in which case the direction is simply reversed.

The equation for eigenvalue is given by,

$\mathbf{A}\,\mathbf{v} = \lambda\,\mathbf{v}$

Where,

- **A** is the matrix,
- **v** is the associated eigenvector, and
- **λ** is the scalar eigenvalue.

Eigenvectors

Eigenvectors are non-zero vectors that, when multiplied by a matrix, only stretch or shrink without changing direction. The eigenvalue must be found first before the eigenvector. For any square matrix **A** of order **n × n**, the eigenvector is a column matrix of size **n × 1**. This is known as the right eigenvector, as matrix multiplication is not commutative.

Alternatively, the left eigenvector can be found using the equation **v A = λ v**, where **v** is a row matrix of size **1 × n**.

Eigenvector Equation

The eigenvector equation is:

A v = λ v

Rewriting the right-hand side:

λ v = λ I v

Then,

A v - λ I v = 0

(A - λ I) v = 0

For a non-zero vector **v**, this implies:

det(A - λ I) = 0

This is known as the **characteristic equation**.

How to Find an Eigenvector?

1. Find the eigenvalues of the matrix **A** using **det(A – λI) = 0**, where **I** is the identity matrix of the same order as **A**.
2. The values obtained are **λ_1, λ_2, λ_3, …**
3. For each eigenvalue **λ_i**, solve **(A – λ_i I) X = 0** to find the corresponding eigenvector **X**.
4. Repeat for all eigenvalues.

Types of Eigenvector

- **Right Eigenvector**: Solved using **A V_R = λ V_R**, where **V_R** is a column vector.
- **Left Eigenvector**: Solved using **V_L A = λ V_L**, where **V_L** is a row vector.

Eigenvectors of a Square Matrix

Eigenvector of a 2 × 2 Matrix

Example: Find the eigenvalues and eigenvectors for the matrix
A = \begin{bmatrix} 1 & 2 \ 5 & 4 \end{bmatrix}

Solution:

Characteristic equation: **det(A - λI) = 0**

[\begin{vmatrix} 1 - \lambda & 2 \ 5 & 4 - \lambda \end{vmatrix} = 0]

(1-λ)(4-λ) - 10 = 0
λ^2 - 5λ - 6 = 0

$(\lambda - 6)(\lambda + 1) = 0$

Eigenvalues: $\lambda = 6$, $\lambda = -1$

For **$\lambda = 6$**:
$(A - 6I)v = 0 \rightarrow 5a - 2b = 0 \rightarrow v = \begin{bmatrix} 2 \ 5 \end{bmatrix}$ (or multiples)

For **$\lambda = -1$**:
$(A + I)v = 0 \rightarrow a + b = 0 \rightarrow v = \begin{bmatrix} 1 \ -1 \end{bmatrix}$ (or multiples)

Eigenvector of a 3 × 3 Matrix

Example: Find the eigenvalues and eigenvectors for the matrix
$A = \begin{bmatrix} 2 & 2 & 2 \ 2 & 2 & 2 \ 2 & 2 & 2 \end{bmatrix}$

Solution:

Characteristic equation leads to **$\lambda = 6, 0, 0$**

Corresponding eigenvectors include:
For $\lambda = 6$: **$\begin{bmatrix} 1 \ 1 \ 1 \end{bmatrix}$**
For $\lambda = 0$: **$\begin{bmatrix} -1 \ 1 \ 0 \end{bmatrix}$**, **$\begin{bmatrix} -1 \ 0 \ 1 \end{bmatrix}$** (and linear combinations)

Here's the **exact content** extracted from the webpage:
https://www.geeksforgeeks.org/data-science/singular-value-decomposition-svd/

Singular Value Decomposition (SVD)

Last Updated : 5 Jul, 2025

Singular Value Decomposition (SVD) is a factorization method in linear algebra that decomposes a matrix into three other matrices, providing a way to represent data in terms of its singular values.

SVD helps you split that table into three parts:

- **U**: This part tells you about the people (like their general preferences).
- **Σ**: This part shows how important each factor is (how much each rating matters).
- **V^T**: This part tells you about the products (how similar they are to each other).

Let's understand this with the help of an example: Suppose you have a small table of people's ratings for two movies,

Name	Movie 1 Rating	Movie 2 Rating
Amit	5	3
Sanket	4	2
Harsh	2	5

- SVD breaks this table into three smaller parts: one that shows people's preferences, one that shows the importance of each movie, and one that shows how similar the movies are to each other.
- Mathematically, the SVD of a matrix **A** (of size **m × n**) is represented as:
  $A = U \Sigma V^T$

Here:

- **U**: An **m × m** orthogonal matrix whose columns are the left singular vectors of **A**.
- **Σ**: A diagonal **m × n** matrix containing the singular values of **A** in descending order.
- **V^T**: The transpose of an **n × n** orthogonal matrix, where the columns are the right singular vectors of **A**.

How to perform Singular Value Decomposition

To perform Singular Value Decomposition (SVD) for the matrix
A = \begin{bmatrix} 3 & 2 & 2 \ 2 & 3 & -2 \end{bmatrix}, let's break it down step by step.

Step 1: Compute A A^T

A = \begin{bmatrix} 3 & 2 & 2 \ 2 & 3 & -2 \end{bmatrix}
A^T = \begin{bmatrix} 3 & 2 \ 2 & 3 \ 2 & -2 \end{bmatrix}

A A^T = \begin{bmatrix} 17 & 8 \ 8 & 17 \end{bmatrix}

Step 2: Find the Eigenvalues of A A^T

Solve the characteristic equation:
det(A A^T - λI) = 0

Eigenvalues: **λ_1 = 25**, **λ_2 = 9**

Singular values: **σ_1 = 5**, **σ_2 = 3** (square roots of the eigenvalues).

Step 3: Find the Right Singular Vectors (Eigenvectors of A^T A)

For **λ = 25**:
v_1 = \begin{bmatrix} \frac{1}{\sqrt{2}} \ \frac{1}{\sqrt{2}} \ 0 \end{bmatrix}

For **λ = 9**:
v_2 = \begin{bmatrix} \frac{1}{\sqrt{18}} \ \frac{-1}{\sqrt{18}} \ \frac{4}{\sqrt{18}} \end{bmatrix}

For the third eigenvector **v_3** (orthogonal to v_1 and v_2):
v_3 = \begin{bmatrix} \frac{2}{3} \ \frac{-2}{3} \ \frac{-1}{3} \end{bmatrix}

Step 4: Compute the Left Singular Vectors (Matrix U)

U = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \ \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{bmatrix}

Step 5: Final SVD Equation

A = U Σ V^T

Where:
U = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \ \frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} \end{bmatrix}
Σ = \begin{bmatrix} 5 & 0 & 0 \ 0 & 3 & 0 \end{bmatrix}
V = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \ \frac{1}{\sqrt{18}} & \frac{-1}{\sqrt{18}} & \frac{4}{\sqrt{18}} \ \frac{2}{3} & \frac{-2}{3} & \frac{1}{3} \end{bmatrix}

Applications of Singular Value Decomposition (SVD)

1. **Calculation of Pseudo-Inverse (Moore-Penrose Inverse)**
   The pseudo-inverse is a generalization of the matrix inverse, applicable to non-invertible matrices like low-rank matrices.

For $\mathbf{M} = \mathbf{U}\,\mathbf{\Sigma}\,\mathbf{V}^T$, the pseudo-inverse is:
$\mathbf{M}^+ = \mathbf{V}\,\mathbf{\Sigma}^{-1}\,\mathbf{U}^T$ (where Σ^{-1} inverts the non-zero singular values).

2. **Solving a Set of Homogeneous Linear Equations**
   For $\mathbf{M}\,\mathbf{x} = \mathbf{b}$:
   - If $\mathbf{b} = \mathbf{0}$, use SVD to choose a column of $\mathbf{V}$ associated with a zero singular value.
   - If $\mathbf{b} \neq \mathbf{0}$, solve $\mathbf{x} = \mathbf{M}^+\,\mathbf{b}$.
3. **Rank, Range, and Null Space**
   - **Rank**: Number of non-zero singular values in $\mathbf{\Sigma}$.
   - **Range**: Span of the left singular vectors in $\mathbf{U}$ corresponding to non-zero singular values.
   - **Null Space**: (The article cuts off here in the source, but this is the listed point.)

Vector Norms

A vector norm, sometimes represented with a double bar as $\|\mathbf{x}\|$, is a function that assigns a non-negative length or size to a vector $\mathbf{x}$ in n-dimensional space. Norms are essential in mathematics and machine learning for measuring vector magnitudes and calculating distances.

A vector norm satisfies three properties:

- **Non-negativity**: $\|x\| > 0$ if $x \neq 0$, and $\|x\| = 0$ if and only if $x = 0$.
- **Scalar Multiplication**: $\|kx\| = |k| \cdot \|x\|$ for any scalar k.
- **Triangle Inequality**: $\|x + y\| \leq \|x\| + \|y\|$.

Types of Vector Norms

The vector norm $\|\mathbf{x}\|_\mathbf{p}$, also known as the **p-norm**, for p = 1, 2, … is defined as:

$$ |\mathbf{x}|p = \left(\sum{i=1}^{n} |x_i|^p \right)^{\frac{1}{p}} $$

This general formula encompasses several specific norms that are commonly used.

Commonly used norms are:

- L1 Norm
- L2 Norm
- L∞ Norm

L1 Norm

The **L1 norm**, also known as the **Manhattan norm** or **Taxicab norm**, is a way to measure the "length" or "magnitude" of a vector by summing the absolute values of its components.

Mathematically, for a vector $\mathbf{x} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n]$, the L1 norm $\|\mathbf{x}\|_1$ is defined as:

$$ |x|_1 = |x_1| + |x_2| + |x_3| + \dots + |x_n| $$

Example: If $\mathbf{x} = [3, -4, 2]$, then the L1 norm is:
$\|\mathbf{x}\|_1 = |3| + |-4| + |2| = 3 + 4 + 2 = 9$

L2 Norm

The **L2 norm**, also known as the **Euclidean norm**, is a measure of the "length" or "magnitude" of a vector, calculated as the square root of the sum of the squares of its components.

For a vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]$, the L2 norm $\|\mathbf{x}\|_2$ is defined as:

$$|x|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

Example: If $\mathbf{x} = [3, -4, 2]$, then the L2 norm is:
$$|x|_2 = \sqrt{3^2 + (-4)^2 + 2^2} = \sqrt{9 + 16 + 4} = \sqrt{29} \approx 5.39$$

L∞ Norm

The **L∞ norm**, also known as the **Infinity norm** or **Max norm**, measures the "size" of a vector by taking the largest absolute value among its components. Unlike the L1 and L2 norms, which consider the combined contribution of all components, the L∞ norm focuses solely on the component with the maximum magnitude.

For a vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]$, the L∞ norm $\|\mathbf{x}\|_\infty$ is defined as:

$$|x|_\infty = \max |x_i| \quad \text{where } 1 \leq i \leq n$$

Example: If $\mathbf{x} = [3, -4, 2]$, then the L∞ norm is:
$\|\mathbf{x}\|_\infty = \max(|3|, |-4|, |2|) = 4$

Practice Problems Based on Vector Norm

Question 1. Given the vector $\mathbf{x} = [4, -3, 7, 1]$, calculate the L1 norm (Manhattan norm) of the vector.

Question 2. Given the vector $\mathbf{x} = [1, -2, 2]$, calculate the L2 norm (Euclidean norm) of the vector.

Question 3. For the vector $\mathbf{x} = [7, -1, -4, 6]$, calculate the L∞ norm (Infinity norm) of the vector.

Question 4. If the L2 norm (Euclidean norm) of a vector $\mathbf{x} = [x_1, x_2, x_3]$ is 10, and the components of the vector are $x_1 = 6$ and $x_2 = 8$, find the value of x_3.

Answers:

1. 15
2. 3
3. 7
4. 0

Measures of Distance

Measures of distance are mathematical functions used to quantify how similar or dissimilar two objects are based on their features. These measures are critical for clustering, classification and information retrieval because they help determine relationships among data points. The choice of distance depends on the nature of the data and the application domain.

- Used to quantify similarity/dissimilarity between objects.
- Smaller distance = higher similarity, larger distance = higher dissimilarity.
- Important for clustering algorithms (e.g., K-means, hierarchical clustering).
- Choice of distance is context-dependent (numerical, categorical or text data).

Let's see few types of distances.

1. Euclidean Distance

Euclidean Distance is considered the traditional metric for problems with geometry. It can be simply explained as the ordinary distance between two points. It is one of the most used algorithms in the cluster analysis.

Formula:

$$ d(x,y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} $$

- **Best for**: Continuous numerical data (when features are normalized).
- **Example**: Distance between two cities on a 2D map.

2. Manhattan Distance

Manhattan Distance determines the absolute difference among the pair of the coordinates. Suppose we have two points P and Q to determine the distance between these points we simply have to calculate the perpendicular distance of the points from X-Axis and Y-Axis. In a plane with P at coordinate (x1, y1) and Q at (x2, y2). Manhattan distance between P and Q = |x1 – x2| + |y1 – y2|

Formula:

$$ d(x,y) = \sum_{i=1}^n |x_i - y_i| $$

- **Best for**: High-dimensional data or when diagonal movement has no meaning.
- **Example**: Distance between blocks in a city (taxicab geometry).

3. Jaccard Index

The Jaccard distance is set-based distance that compares dissimilarity by looking at the ratio of unique to common elements.

Formula:

$$ d(A,B) = 1 - \frac{|A \cap B|}{|A \cup B|} $$

- **Best for**: Binary or categorical data, especially sets.
- **Example**: Comparing similarity of shopping carts or tag sets.

4. Minkowski Distance

Minkowski distance is a generalized distance measure that includes both Euclidean and Manhattan distances as special cases, controlled by a parameter p.

Formula:

$$ d(x,y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}} $$

- **Best for**: Flexible distance calculations where p is tuned.
- **Example**: For p=1, it becomes Manhattan; for p=2, it becomes Euclidean.

5. Cosine Similarity / Cosine Distance

Measures the cosine distance of the angle between two vectors, focusing on orientation rather than magnitude. Commonly converted to distance as 1−similarity.

Formula (Similarity):

$$ \text{Cosine}(x,y) = \frac{x \cdot y}{||x|| , ||y||} $$

Formula (Distance):

$$ d(x,y) = 1 - \frac{x \cdot y}{||x|| , ||y||} $$

- **Best for**: Text mining, NLP, recommendation systems.
- **Example**: Measuring similarity between two documents regardless of their length.

🔗6. Hamming Distance

The number of positions where two strings (of equal length) differ. Commonly used for error detection and sequence comparison.

Formula:

$$ d(x,y) = \sum_{i=1}^n [x_i \neq y_i] $$

where $[x_i \neq y_i] = 1$ if symbols differ, else 0.

- **Best for**: Binary strings, DNA sequences, error correction.
- **Example**: Hamming distance between “karolin” and “kathrin” = 3.